Buffer Overflow Assignment

(Due: July 8)

Address randomization. In this lab assignment, you will experience the buffer overflow attack. When we execute a program, Ubuntu and some other Linux-based systems uses address space randomization to randomize the starting address of heap and stack. This makes guessing the exact addresses difficult. But guessing addresses is one of the critical steps of buffer-overflow attacks. So we need to disable it, using the following command: `$ sudo sysctl -w kernel.randomize_va_space=0`

Configure the Shell program. In recent Ubuntu OS, **/bin/sh** symbolic link points to the **/bin/dash** shell. The dash program, as well as bash, has implemented a security countermeasure that prevents itself from being executed in a Set-UID process. Basically, if they detect that they are executed in a Set-UID process, they will immediately change the effective user ID to the process's real user ID essentially dropping the privileges. Our victim program is Set-UID program. This counter measure makes us hard, if not impossible, to run this program on /bin/dash. We thus change /bin/sh to point to another shell program zsh (that does not have this countermeasure), using the following command:

```
$ sudo ln -sf /bin/zsh /bin/sh
```

Compilation. To compile our vulnerable program, we need to turn two stack protections, using "-fno-stack-protector" and "-z execstack". Also, we need to make the output program root-owned and Set-UID. These can be done as follows.

```
$ gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector stack.c
$ sudo chown root stack          ①
$ sudo chmod 4755 stack          ②
```

Here command 1 makes the executable ***stack*** root-owned and command 2 makes it Set-UID. That is, when run by an ordinary user, the effect process ID will be root and the process owner will be this ordinary user. The -m32 means that the output program (i.e., stack) is for x86 architecture system and without it the output program is for x64 architecture. Your machine should work for both options.

BUF_SIZE is the option parameter for the size of **buffer** in the bof() function. Once we set it in gcc, bof will use this because BUF_SIZE in stack.c uses **ifndef BUF_SIZE**. The BUF_SiZE is defined in the Makefile as L1, L2, L3, L4 choices.

Problem 1. (**Buffer Overflow Attack on 32-bit Program**) The vulnerable program is stack.c. Compile it to **stack-L1**, using **make stack-L1** at the Labsetup/code directory, where Makefile is already constructed. The stack.c takes badfile as an input. You need to create this badfile so that when you execute stack-L1, it gives you a zsh shell. The code (I call it *starting-zsh* code) for starting a zsh shell is the following.

"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"

"\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"

"\xd2\x31\xc0\xb0\x0b\xcd\x80"

We will embed it into badfile. The badfile will be generated by exploit.py. So please copy it into **shellcode** variable in exploit.py. To generate a working badfile, you need to further find **start**, **RET** and

offset three variables. **start** is the byte position of **content** (or **buffer** in bof() when later copied from content), where the above starting-zsh code will begin. You can choose this freely but make sure that start+27(code length)<517 (content length). Also, you need to make sure **start**>RET_holder_address-**buffer (**the first byte address of **buffer),** where RET_holder_address is the address location in buffer that holds RET (i.e., the return address of bof()). In a debug mode, you can find frame pointer ebp of bof using command **p $ebp**. This is the address that holding the previous frame pointer (i.e., the frame pointer of dummy_function). $ebp+4 will be RET_holder_address and **offset**=4+**$ebp**-**buffer** (the first byte address of buffer) and this offset is stable for different executions of **stack-L1** and so you can obtain using debug **stack-L1-dbg**. But $ebp and buffer respectively might change from different executions. **RET** is the return address when bof finishes the execution and it is stored at the RET_holder_address. In our attack, RET is the address of the malicious code in buffer. So ideally, you can set exactly as **RET**=**buffer**+**start=$ebp+(start-offset)**. However, keep in mind that $ebp in your execution ./stack-L1 could be bigger or smaller than that you obtain in the debug mode. But this difference is not much (e.g., <50). However, we only need to make sure RET lies in the NOP sled between RET_holder_address and buffer+start. This gives you some flexibility. Please complete the following.

- Detail how you choose **start**, **RET** and **offset**.
- Us the screenshot to witness your choice.
- Show your successful execution screenshot of your experiment.

/* Note: this problem is alternative description of Task 3 in Buffer_Overflow_SetUID.pdf. Feel free to consult it if you need. ***/**

Problem 2. (defeat dash countermeasure) The dash shell in the Ubuntu OS drops privileges when it detects that the effective UID does not equal to the real UID (which is the case in a Set-UID program). This is achieved by changing the effective UID back to the real UID (see the code below).

```
++  uid = getuid();
++  gid = getgid();

++  /*
++   * To limit bogus system(3) or popen(3) calls in setuid binaries,
++   * require -p flag to work in this situation.
++   */
++  if (!pflag && (uid != geteuid() || gid != getegid())) {  ①
++     setuid(uid);
++     setgid(gid);
++     /* PS1 might need to be changed accordingly. */
++     choose_ps1();
++  }
```

To defeat this protection, the idea is to set the uid to be 0 (root) in the code. The c program for this purpose can be seen as follows.

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
     char *argv[2];
     argv[0] = "/bin/sh";
     argv[1] = NULL;

     setuid(0);  // Set real UID to 0
     execve("/bin/sh", argv, NULL);

     return 0;
```

We can update the attack code by including setuid(0). Under this, dash shell will regard the execution user as root and so the shell will be deceptively started. The modified shell code is the following:

```
"\x31\xc0\x31\xdb\xb0\xd5\xcd\x80\x31\xc0"
"\x50\x68""//sh""\x68""/bin""\x89\xe3\x50"
"\x53\x89\xe1\x99\xb0\x0b\xcd\x80"
```

Replace this code with the code in exploit.py and run it against **stack-L1** again. But you need to go through the similar procedure in problem 1 to determine **start**, **offset** and **ret**. Do the following.

- Detail how you choose start, RET and offset.
- Us the screenshot to witness your choice.
- Show your successful execution screenshot of your experiment.

/* Note: The description of problem is based on Task 7 in the Buffer_Overflow_SetUID.pdf. Feel free to consult it if you need. ***/**

Problem 3. (**Defeat Address Randomization**) The buffer overflow attack needs to guess the absolute address (although not accurately) of some address on the stack. This is needed for **RET**. This is based on the stackframe base address. If we can randomize this address, then **RET** is randomized and can not be predetermined and written into RET holder. This randomized address is determined by the loader program of OS because it is loader's job to determine the locations of stackframe, heap and more. In Linux, the level of address randomization is set via parameter: **kernel.randomize_va_space.** Here, kernel.randomize_va_space=0 means no randomization; 1 means stackframe is randomized while heap has no drandomization; 2 means that both are randomized. However, even if full randomization is set, the randomization entropy is only 19 and so the search space is only 2^{19}= 524288. We can use the following program is to repeatedly run the program **stack-L1** and wait for the instance that uses the randomization that matches our badfile, in which case we succeed. This attack is effective against 32-bit program **stack-L1**. The attack code is **brute-force.sh.** Execute it and show the screenshot of the success. For details for this experiment, you can consult Section 4.9 in sample_buffer_overflow.pdf.

Problem 4 (optional**)**. (**attack 64-bit program**) Problem 1 has attacked 32-bit program **stack-L1**. In the current problem, we will attack the 64-bit program, compiled by **make stack-L3** (see Makefile for stack-L3 option, where L3=200 and -m32 is removed). Compared to buffer-overflow attacks on 32-bit architecture, the attack on 64-bit architecture is more difficult. The most difficult part comes from the address format. Although the x64 architecture supports 64-bit address space, only the address from **0x00 through 0x00007FFFFFFFFFFF** is allowed. That means for every address (8 bytes), the highest two bytes are always zeros. This causes a problem. In our buffer-overflow attacks, we need to store at least one address (i.e. **RET**) into badfile, and it then will be read into **str** and then copied to **buffer** via strcpy(). We know that the strcpy() function will stop copying when it sees byte zero. Therefore, if byte zero appears in the middle of badfile, the content after this zero cannot be copied (from **str** to **buffer**). This is important to us because the spot holding **RET** in badfile in problem 1 presented prior to our malicious code while now RET contains byte zero so the malicious code will not be copied into **buffer**. So we need a different arrangement of the badfile. The idea is to move the malicious code to a location prior to the spot holding **RET**. This is not enough because if RET=**0x00007ffffff0abc** is stored in the memory in this format, then the copying will stop at byte \x00. It will be good if RET is stored as **bc 0a ff ff ff 7f 00 00** because 00 appears the last and 00 copied or not does not matter as any address in x64 system ended with 00 00 (especially it is true for the old RET value in the RET holder in bof stack frame, to be overwritten by our fake RET). This storage format is called little endian and is actually what many Linux

systems use. So we do not need to bother with this issue. In this attack, you need to figure out **start**, **offset** and **ret** in exploit.py and execute it to output badfile. The shellcode needs to use the following:

"\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
"\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
"\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"

To find **start**, **offset** and **ret**, you can still use the strategy similar to problem 1. But keep in mind that the frame pointer in x64 is rbp and the **RET** holder address is rbp+**8** as it is 64-bit machine, as one address has 8 bytes now. The requirement of this problem is as follows.

- Detail how you choose **start**, **RET** and **offset**.
- Us the screenshot to witness your choice.
- Show your successful execution screenshot of your experiment.

/* Note: The description of problem is based on Task 5 in the Buffer_Overflow_SetUID.pdf. Feel free to consult it if you need. ***/**

Problem 5 (optional**)**. (**Attack 64-bit Program with buffer size 10**) In the previous problem, we know the x64 architecture has the address format starting with \x0000. So the attack must put the RET to the end of the badfile and put the malicious code prior to this. That requires the **buffer** variable in bof() to have a size of malicious-code length (i.e. 30 in our case). In the current problem, **buffer** has a length 10 only and it is not enough to hold the malicious code. Our attack target is the 64-bit program **stack-L4**. It can be generated by **make stack-L4**. For 32-bit program, this is not an issue because the shellcode is always placed in the overflow region (beyond RET holder). So the difficulty remains only for our 64-bit program **stack-L4**, due to \x0000 issue. We need a new strategy. The idea is as follows. Remember that badfile is first read into **str** in **main**() function. The variable **str** is stored on the stackframe of main. Later in bof(), we do strcpy() which try to copy **str** to **buffer**. But **RET_holder_address** - **buffer** is small (less than 20) and **RET_holder_address** is ending with \x0000 and the malicious code in **str** will not be copied to **buffer**. However, it is till stored in **str**. If we can find the address of malicious code in **str**, we can set it as **RET** and written into **RET_holder_address** in bof stack frame. Actually, you do not have find the precise address of **RET**. It suffices to find some smaller address because we use NOP to lead the execution to the malicious code. This will be useful because due to the instance differences the precise malicious code address is unlikely to be found precisely. For our work, complete the following.

- Detail how you choose **start**, **RET** and **offset**.
- Us the screenshot to witness your choice.
- Show your successful execution screenshot of your experiment.

/* Note: The description of problem is based on Task 6 in the Buffer_Overflow_SetUID.pdf. Feel free to consult it if you need. ***/**